

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News • [R Resources](#) • > •

Social Networks in R

I use social network analysis as a tool for understanding the social relationships between individuals in some avian systems. I am in the process of learning to use two packages for social network analysis in R--[statnet](#) and [igraph](#)--and I have decided to post what I have learned in hopes that someone else might find this information useful. Below are small tutorials I have written up for various aspects of social network analysis in R. I hope you find some of this useful.

The codes here have been updated for igraph v0.6

Subpage Listing

01. [igraph vs statnet](#)

02. Importing data for social network analysis

03. Affiliation Networks/Bipartite Networks

04. Node-level Calculations

05. Plotting networks - Basics

06. Plotting networks: using node attributes

07. Plotting Networks: Weighted Edges

08. Plotting Networks: Directed Networks

09. Edgelists with edge values

10. Raw dyadic interaction data

Association Indices

sexualnetworks

Triangle Transitivity in dominance hierarchies & directed graphs

Subpages (13): [01. igraph vs statnet](#) [02. Importing data for social network analysis](#) [03. Affiliation Networks/Bipartite Networks](#) [04. Node-level Calculations](#) [05. Plotting networks - Basics](#) [06. Plotting networks: using node attributes](#) [07. Plotting Networks: Weighted Edges](#) [08. Plotting Networks: Directed Networks](#) [09. Edgelists with edge values](#) [10. Raw dyadic interaction data Association Indices sexualnetworks](#) [Triangle Transitivity in dominance hierarchies & directed graphs](#)

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)
- [People](#)
- [Research](#)
- [Publications](#)
- [R Resources](#)
- [Teaching](#)
- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

01. igraph vs statnet

There are [many](#) software packages out there that allow you to conduct social network analysis, including the widely-used [UCINET](#) and [PAJEK](#) programs. However, because network analysis is an ever-evolving field, using a platform like R that allows flexibility and ever-increasing functions has its advantages.

There are two packages that allow you to conduct social network analysis in R: [igraph](#) and [statnet](#). Another package I may play with is [tnet](#), which is made to calculate measures from weighted networks. As with anything else, there are pros and cons to each package. For the basics, I will often present ways to do things in both. I will also attempt to keep track of general pros & cons (as applies to me) of each package.

General information:

[statnet](#)

- *statnet* is actually a suite of packages that have been developed by that [statnet development team](#) based out of University of Washington. It allows you to statistically test models based on an Exponential-Family Random Graph Model framework. In fact, statistical modeling is the explicit purpose for the development of this package.

- However, two of the packages that are included in the *statnet* suite, *sna* and *network*, provides more basic functions of network analysis, and that is what I'll be using most of the time. As such, if you are looking for basic functions, you should look at the documentation for *sna* and *network* (in R, use `library(help=sna)` or `library(help=network)`). But installing one package is easier, just install *statnet* and all its dependencies
- The biggest advantage of *statnet* is that it is expansive and probably allows you to do most things you might want to do in social network analysis. The trade-off, of course, is that because it is so comprehensive, it can be daunting and complicated.

igraph

- *igraph* is a somewhat more basic approach to network analysis. However, it still contains a lot of functionality, including calculating network properties, generating random graphs for simulations, etc. and will probably fit most of your needs.
- *igraph* seems to be more efficient than *statnet*, and the many of the functions (particularly manipulating data and dealing with vertex attributes) seem more intuitive to me.
- *igraph* has a nice tutorial online that is in development. You can find it [here](#). It is very helpful.

More on social networks in R

-

[01. igraph vs statnet](#)

-

[02. Importing data for social network analysis](#)

-

[03. Affiliation Networks/Bipartite Networks](#)

-

[04. Node-level Calculations](#)

-

[05. Plotting networks - Basics](#)

-

[06. Plotting networks: using node attributes](#)

-

07. Plotting Networks: Weighted Edges

-

08. Plotting Networks: Directed Networks

-

09. Edgelist with edge values

-

10. Raw dyadic interaction data

-

Association Indices

-

sexualnetworks

-

Triangle Transitivity in dominance hierarchies & directed graphs

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- Shizuka Lab Home

- People

- Research

- Publications

- R Resources

- Teaching

- News

News R Resources • > • Social Networks in R • > •

02. Importing data for social network analysis

**** This page is now updated for igraph version 0.6.3 in R 2.15.2 (Feb 2, 2013)**

In both statnet and igraph, you start with importing a dataset, which you convert into either a "network object" (statnet) or "igraph object" (igraph). Basically, it's just telling R to recognize a given set of numbers as something to be manipulated in the network analysis framework.

There are several formats in which you can save data for use in social network analysis, (1) adjacency matrix, (2) edge lists, (3) adjacency lists (or node lists) and (4) affiliation matrix/incidence matrix. I will talk about affiliation matrices in a [separate section for bipartite networks](#).

(1) An **adjacency matrix** is a matrix in which the rows and columns represent different *nodes* or *vertices* (i.e., the dots on the sociogram). In an unweighted adjacency matrix, the *edges* (i.e., lines) are represented by 0 or 1, with indicating that these two nodes are connected. If two nodes are connected, they are said to be adjacent (hence the name, adjacency matrix). In a weighted matrix, however, you can have different values, indicating different edge qualities (or tie strengths). Here is an example of a hypothetical adjacency matrix of birds with 5-digit ID numbers.

	23732	23778	23824	23871	58009	58098	58256
23732	0	1	0	1	0	1	0

23778	1	0	1	1	0	1	0
23824	0	1	0	0	0	0	0
23871	1	1	0	0	1	1	0
58009	0	0	0	1	0	1	0
58098	1	1	0	1	1	0	1
58256	0	0	0	0	0	1	0

You can import an adjacency matrix into a usable format in either statnet or igraph using these sample codes (note, this is for an unweighted matrix):

in igraph:

```
library(igraph) # This loads the igraph package
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # choose an adjacency matrix from a .
csv file
m=as.matrix(dat) # coerces the data set as a matrix
g=graph.adjacency(m,mode="undirected",weighted=NULL) # this will create an 'igraph object'
g
```

In igraph version 0.6, the way the igraph object is displayed has changed. The above code will return:

```
IGRAPH UN-- 7 10 --
+ attr: name (v/c)
```

The letters on the first line (there can be up to 4) indicates some basic information about the graph. The first letter indicates whether this is a directed ('D') or undirected ('U') graph. The 2nd letter tells you if this is a named ('N') graph--i.e., whether or not the vertex set has a 'name' attribute. The 3rd letter tells you if this graph is weighted ('W'). The fourth letter is 'B' for bipartite graphs. These letter codes are followed by two numbers: the first is the number of vertices and the second is the number of edges.

The second line gives you information about the 'attributes' associated with the graph. In this case, there is only one attribute, called 'name', which is associated with the vertex set.

equivalent in statnet:

```
library(statnet) # This loads the statnet package
el=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # choose an adjacency matrix from a .
csv file
m=as.matrix(el) # This coerces the object into a matrix, just in case
net=network(m,matrix.type="adjacency",directed=FALSE) # This converts the matrix into a an undirected
"network object"
```

(2) An **edge list** is a two-column list of the two nodes that are connected in a network. Here's the same data as above, but in an edge list form.

V1	V2
23732	23778

23732	23871
23732	58098
23778	23824
23778	23871
23778	58098
23871	58009
23871	58098
58009	58098
58098	58256

****You can also easily input edgelists in igraph or statnet. HOWEVER, there is one major annoyance with statnet: it tends to read the numbers as continuous numbers, inputting this list directly will produce a network with 58256 nodes (the highest number)--most of which will not be connected. The code provided below gets around the problem, but it was very annoying at first. You will not have this problem if your node IDs are not numerical.**

The simplest workaround for this is to make sure that the IDs are read as characters:

in igraph:

```
library(igraph)
```

```
dat=read.csv(file.choose(),header=TRUE) # choose an edgelist in .csv file format
el=as.matrix(dat) # coerces the data into a two-column matrix format that igraph likes
el[,1]=as.character(el[,1])
el[,2]=as.character(el[,2])
g=graph.edgelist(el,directed=FALSE) # turns the edgelist into a 'graph object'
```

****A much easier way to import an edge list in igraph** is to use the `graph.data.frame()` function:

```
g=graph.data.frame(dat,directed=FALSE)
```

in statnet:

```
el=read.csv(file.choose(),header=TRUE) # read a .csv file
el[,1]=as.character(el[,1])
el[,2]=as.character(el[,2])
n=network(el,matrix.type="edgelist",directed=FALSE)
```

(3) An **adjacency list**, also known as a node list, presents the 'focal' node on the first column, and then all the other nodes that are connected to it (i.e., adjacent to it) as columns to the right of it.

Focal	Interactor 1	Interactor 2	Interactor 3	Interactor 4	Interactor 5
23732	23778	23871	58098		
23778	23732	23824	23871	58098	
23824	23778				
23871	23732	23778	58009	58098	
58009	23871	58098			
58098	23732	23778	23871	58009	58256

58256

58098

This is sometimes a convenient way to organize the data from the field... However, the major downside is that it is not very easy to import data this way into igraph or statnet. igraph DOES support importing an adjacency list into a graph object, but you need the data to be in a specific format (a 'list object', with row as a different vector)... I can't get this to work. The best way to deal with it seems to be to use the codes below to convert the adjacency list into an edgelist. I'm going to assume the data is in the format as shown above, with the first row being column names, and saved as a .csv file:

```
lines=scan(file.choose(),what="character",sep="\n",skip=1) # read the csv file (skipping the header), line-
by-line as character string.
lines=gsub(","," ",lines) # replace commas with spaces
lines=gsub("[ ]+$","",gsub("[ ]+","",lines)) # remove trailing and multiple spaces.
adjlist=strsplit(lines," ") # splits the character strings into list with different vector for each line
col1=unlist(lapply(adjlist,function(x) rep(x[1],length(x)-1))) # establish first column of edgelist by
replicating the 1st element (=ID number) by the length of the line minus 1 (itself)
col2=unlist(lapply(adjlist,"[",-1)) # the second line I actually don't fully understand this command, but
it takes the rest of the ID numbers in the character string and transposes it to list vertically
el=cbind(col1,col2) # creates the edgelist by combining column 1 and 2.
```

.... now you have an edgelist, saved as "el" that you can import into igraph or statnet using the codes provided above in the "edgelist" section.

... and here's a set of codes to convert adjacency lists into adjacency matrix, written by Dave McDonald.

****Note:** I have received some feedback indicating that the following code produces errors when copied & pasted. I'm not sure if it's something wonky with how the text appears on the website... In any case, I now provide a text file you can download at the bottom of the page, just in case.

```
dat=scan(file.choose(), what="character",sep="\n")
# scan in a file with IDs as character strings {even if numeric}, and with fields separated by commas;
sample input file is "sample_adjlist.txt"
dym=length(dat)
# stores the number of lines in the file
dat=gsub(","," ",dat)
# convert commas to spaces and add a space {in case the input file lacks any spaces after a comma}
dat=gsub("[ ]+$","",gsub("[ ]+","",dat));
# remove trailing and multiple spaces
m<-matrix(data=0,nrow=dym,ncol=dym)
# create a dymXdym matrix of zeros
adjlist=strsplit(dat," ")
# split the file into a list of length "dym" with distinct elements. N.B. adjlist[[1]][1] calls the first
element in the vector of characters in adjlist

ids=adjlist[[1]][1]
# creates ids of type character to initiate header list of node IDs
for (i in 2:dym) {ids=c(ids,adjlist[[i]][1])}
# creates a character object with the list names
for (i in 1:dym) {adjlist[[i]]=adjlist[[i]][2:length(adjlist[[i]])]}
# strips the first column {which is now in "ids"} from adjlist

tfmat=adjlist
for(i in 1:dym) {tfmat[[i]]<-ids %in% tfmat[[i]]}
# converts tfmat into lists of T-F of length dym of which elements of adjlist[[i]] occur in the ids header
```

```

for(j in 1:dym){for(i in 1:dym){if(tfmat[[j]][i]==T){m[j,i]<-1}}}
# turns cells of matrix m, corresponding to cells in tfmat having value T, into 1s

adjmat=data.frame(m,row.names=ids,check.names=F)
# creates a dataframe "adjmat" with 0/1 values from m and row names from "ids"
names(adjmat)<-ids
# changes header names to the "ids" vector of ID values
adjmat
ls()

```

(4) An **affiliation network** is one in which edges are defined by *co-membership in groups*. Many social networks are defined this way. Conceptually, this ends up being a form of **bipartite graph**. Thus, I will discuss these at length in a [separate page](#) called "affiliation/bipartite networks".

More on social networks in R

-
- 01. [igraph vs statnet](#)
-
- 02. [Importing data for social network analysis](#)
-
- 03. [Affiliation Networks/Bipartite Networks](#)
-
- 04. [Node-level Calculations](#)
-
- 05. [Plotting networks - Basics](#)
-
- 06. [Plotting networks: using node attributes](#)
-
- 07. [Plotting Networks: Weighted Edges](#)
-
- 08. [Plotting Networks: Directed Networks](#)
-

09. Edgelist with edge values

-

10. Raw dyadic interaction data

-

Association Indices

-

sexualnetworks

-

Triangle Transitivity in dominance hierarchies & directed graphs

McD_adjlist_to_adjmat.txt

(2k)

Dai Shizuka,

Aug 2, 2011, 2:13 PM

sample_adjlist.csv

(0k)

Dai Shizuka,

Apr 5, 2010, 5:21 PM

sample_adjmatrix.csv

(0k)

Dai Shizuka,

Apr 5, 2010, 5:21 PM

sample_edgelist.csv

(0k)

Dai Shizuka,

Apr 5, 2010, 5:45 PM

Comments

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)
- [People](#)
- [Research](#)
- [Publications](#)
- [R Resources](#)
- [Teaching](#)
- [News](#)

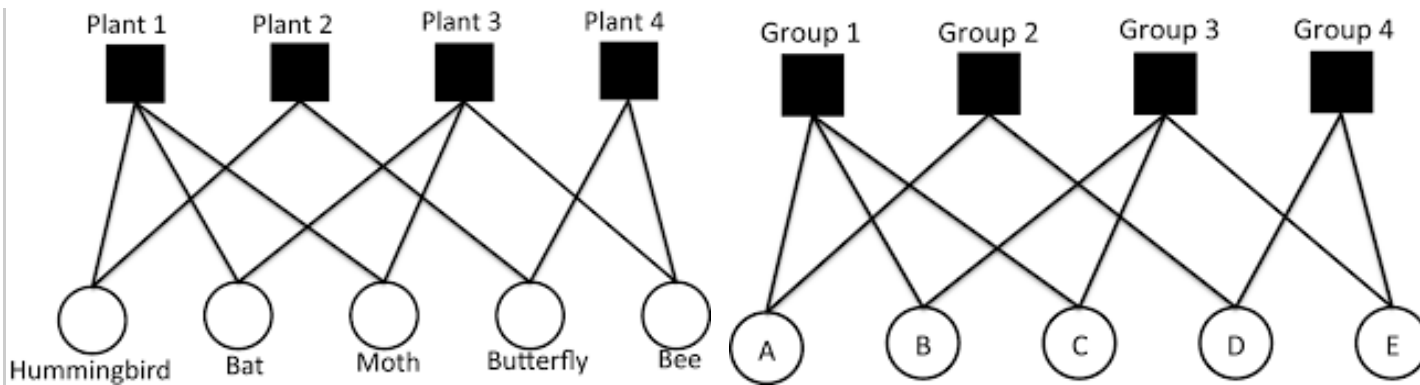
News

[R Resources](#) • > • [Social Networks in R](#) • > •

03. Affiliation Networks/Bipartite Networks

**** This page is now updated for igraph version 0.6.3 in R 2.15.2 (Feb 2, 2013)**

Here, we are going to deal with a slightly different kind of networks, called **bipartite** networks. Here are two examples of bipartite networks:



These two graphs are the same, but they represent two different scenarios.

The first scenario is if we have two types of actors, and one type of actor can only interact with the other type of actor. For example, picture a network of pollination interactions between plants and their pollinators. Pollinators only pollinate plants, so there are no edges between pollinators. Similarly, there are no edges between plants.

The second scenario is when you have individuals belong to particular groups. Edges represent membership of individuals in groups. These are sometimes called **affiliation** networks. Here, you can imagine 'individuals' being one set of nodes, and 'groups' being another set of nodes. Individuals can only be connected to groups, and vice versa. In this case, you might actually be interested in generating a social network of individuals based on co-membership. We can do this using what is called a *one-mode projection*, or *bipartite projection*. We will touch on this a little later.

We'll use the example of 5 individuals belonging to 4 different groups.

```
A=c(1,1,0,0)
B=c(1,0,1,0)
C=c(1,0,1,0)
D=c(0,1,0,1)
E=c(0,0,1,1)
bm=matrix(c(A,B,C,D,E),nrow=5,byrow=TRUE)
dimnames(bm)=list(c("A","B","C","D","E"),c("Group1","Group2","Group3","Group4"))
bm
```

This should create a matrix that looks like this:

```
> bm
  Group1 Group2 Group3 Group4
A      1      1      0      0
B      1      0      1      0
C      1      0      1      0
D      0      1      0      1
E      0      0      1      1
```

You can convert this directly into an igraph object:

```
bg=graph.incidence(bm)
> bg
IGRAPH UN-B 9 10 --
+ attr: type (v/x), name (v/c)
```

From the first line, we see that we have created an **U**ndirected, **N**amed graph that is **B**ipartite and has **9** nodes and **10** edges. The second line tells us that there are two vertex attributes: type and name. Let's check these out:

```
V(bg)$type
V(bg)$name
```

```
> V(bg)$type
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE
> V(bg)$name
[1] "A" "B" "C" "D" "E" "Group1" "Group2"
[8] "Group3" "Group4"
```

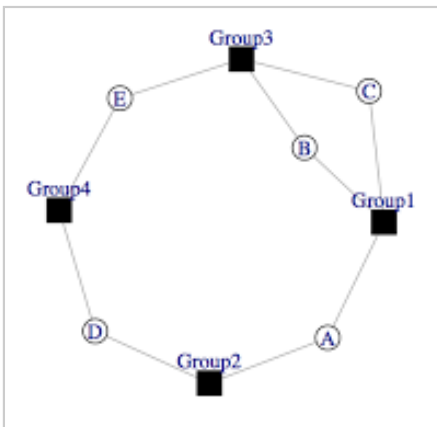
You see that igraph has arranged both individuals and groups as nodes, and then created the attribute 'type' to indicate that these are two distinct classes of nodes.

Now let's see what this network looks like:

```
shapes=c("circle","circle","circle","circle","circle","square","square","square","square") # make the
first 5 nodes circles, and the rest squares.

labeldistances=c(0,0,0,0,0,0.6,0.6,0.6,0.6) #The labels for the first 5 nodes will be in the center of the
node, but the others will be offset

plot(bg,vertex.shape=shapes,vertex.label.degree=-pi/2,vertex.label.dist=labeldistances,vertex.color=V(bg)
$type) #The vertex.label.degree= argument tells where to put the label, in radians.
```



You can see that this network is the same as the second example I showed at the top of this page.

-----Bipartite Projections-----

Let's say you are interested in the co-membership relations between the individuals. What you need to do is to create a *one-mode projection* of the bipartite network. Note that there are actually TWO ways to project a bipartite network: You can make a co-membership network of nodes, or a network of groups that share members. You can get both of these at once with this function:

```
pr=bipartite.projection(bg)
```



```
pr
```

which will give you:

```
> pr
$proj1
IGRAPH UNWB 5 7 --
+ attr: type (v/x), name (v/c), weight (e/n)
```

```
$proj2
IGRAPH UNWB 4 4 --
+ attr: type (v/x), name (v/c), weight (e/n)
```

you can see here that the bipartite projection has given us a list object with two graphs: `pr$proj1` (5 vertices and 6 edges) and `pr$proj2` (4 vertices and 6 edges).

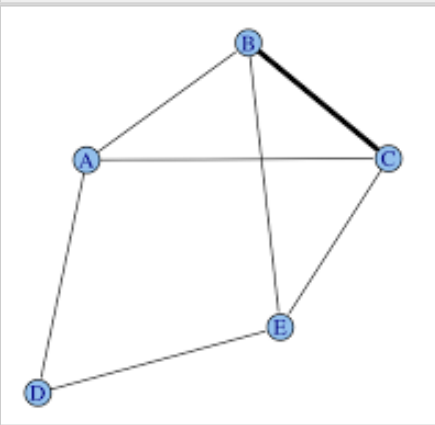
```
get.adjacency(pr$proj1,sparse=FALSE,attr="weight")
```

will give you the adjacency matrix of the first projection:

```
  A B C D E
A 0 1 1 1 0
B 1 0 2 0 1
C 1 2 0 0 1
D 1 0 0 0 1
E 0 1 1 1 0
```

You can plot this projection:

```
plot(pr$proj1,edge.width=E(pr$proj1)$weight^2,edge.color="black",vertex.label=V(pr$proj1)$name)
```



Now, you have an **affiliation network** of individuals based on co-membership in groups.

... at this point, it is useful to know that bipartite projections are not magic. In fact, you can project the bipartite network manually by **multiplying the bipartite / affiliation matrix by its transpose (the same matrix flipped along its diagonal), then setting the diagonal to 0:**

```
aff=bm%*%t(bm)
diag(aff)=0
```

```
> att
  A B C D E
A 0 1 1 1 0
B 1 0 2 0 1
C 1 2 0 0 1
D 1 0 0 0 1
E 0 1 1 1 0
```

You can see that this matrix is identical to the first projection of the bipartite matrix shown above.

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)
- [People](#)
- [Research](#)
- [Publications](#)
- [R Resources](#)
- [Teaching](#)
- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

04. Node-level Calculations

There are certain pre-packaged commands in statnet and igraph that allows you to calculate various node-level measures. The statnet package seems to have a more comprehensive list, though igraph has a couple of measures that statnet does not have. The biggest problems (for my purposes) are that igraph does not have a command for calculating **information centrality**, and neither package seems to have commands for **reach** or **distance-weighted reach**. The latter two are pretty straight-forward, so I am posting functions that will let you easily calculate those two measures.

Here is a list of commands for node-level calculations included in the two packages. To find out what algorithms they use and how to use the commands, just look at the documentation with the help command for each index (**?degree** for example).

Node-level index	statnet	igraph
degree	degree()	degree()

betweenness	betweenness()	betweenness()
power (Bonacich 1987)	bonpow()	bonpow()
closeness	closeness()	closeness()
eigenvector centrality	evcent()	evcent()
flow betweenness	flowbet()	N/A
graph centrality (Harary)	graphcent()	N/A
information centrality	infocent()	N/A
load centrality	loadcent()	N/A
prestige	prestige()	N/A
stress centrality	stresscent()	N/A
reach	see codes below	see codes below
distance-weighted reach	see codes below	see codes below
alpha centrality	N/A	alpha.centrality()
Kleinberg's authority score	N/A	authority.score()
Kleinberg's hub score	N/A	hub.score()

Reach and Distance-weighted Reach

For igraph:

Reach: 2-reach and 3-reach is simply the proportion of nodes you can reach within 2 steps or 3 steps, respectively. Here are codes for calculating this in igraph:

****Update 3/17/2015: I had forgotten to update these codes after igraph v0.6 updates. These should be now fixed.**

2-reach:

```
reach2=function(x){
r=vector( length=vcount(x))
for (i in 1:vcount(x)){
n=neighborhood(x,2,nodes=i)
ni=unlist(n)
l=length(ni)
r[i]=(l)/vcount(x)}
r}

```

3-reach:

```
reach3=function(x){
r=vector( length=vcount(x))
for (i in 1:vcount(x)){
n=neighborhood(x,3,nodes=i)
ni=unlist(n)
l=length(ni)
r[i]=(l)/vcount(x)}
r}

```

distance-weighted reach:

```
dwreach=function(x){
distances=shortest.paths(x) #create matrix of geodesic distances
diag(distances)=1 # replace the diagonal with 1s
weights=1/distances # take the reciprocal of distances
apply(weights,1,sum) # sum for each node (row)
}

```

More on social networks in R

-
- 01. [igraph vs statnet](#)
-
- 02. [Importing data for social network analysis](#)
-
- 03. [Affiliation Networks/Bipartite Networks](#)
-
- 04. [Node-level Calculations](#)
-
- 05. [Plotting networks - Basics](#)
-

06. Plotting networks: using node attributes

-

07. Plotting Networks: Weighted Edges

-

08. Plotting Networks: Directed Networks

-

09. Edgelist with edge values

-

10. Raw dyadic interaction data

-

Association Indices

-

sexualnetworks

-

Triangle Transitivity in dominance hierarchies & directed graphs

Comments

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

05. Plotting networks - Basics

****This page was updated on Feb 3, 2013, using igraph 0.6 on R 2.15**

Note of caution:

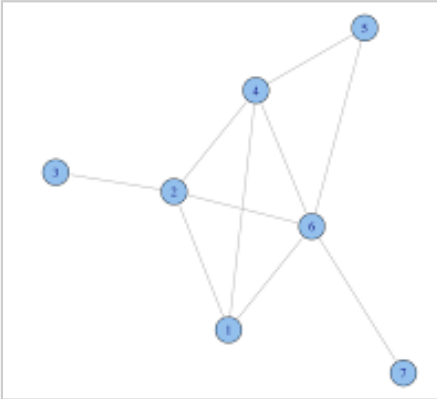
One of the greatest benefits of organizing your data as a network is the ability to visualize the connections between the individuals (or whatever your nodes may be). In some cases, the power of pattern detection by the human eye may be extremely helpful in motivating further questions. However, a great caution is needed—**such visualization exercises can potentially be misleading**. This is at least in part due to our natural tendency to ascribe meaning to the spatial location of the nodes. However, in actual practice, the fact that one node is to the left or right of another node, or even that one node appears at the center of a cluster of other nodes, may mean nothing. It all depends on what algorithm you use, and the parameters you set in producing these figures. The problem is analogous to phylogenetic trees, in which any monophyletic groups can be flipped around at their common ancestor.

With that said, given a clear hypothesis for what you are looking for, you will likely be able to produce a beautiful figure that illustrates those patterns by using the appropriate algorithms. Both statnet and igraph give you this flexibility.

Let's start with the simple default figure, using a sample adjacency matrix (**provided at bottom of page**):

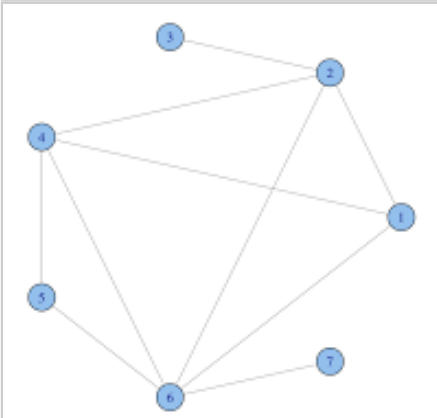
----- In igraph -----

```
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
g=graph.adjacency(m,mode="undirected",weighted=NULL,diag=FALSE)
plot.igraph(g)
```



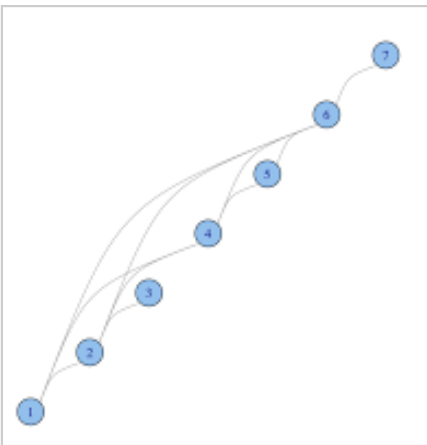
For graphs with <100 nodes, the default layout in igraph 0.6 is now the Kamada-Kawai method, which uses a force-based algorithm. You can change the layout by specifying different algorithms. For example:

```
plot.igraph(g,layout=layout.circle)
```



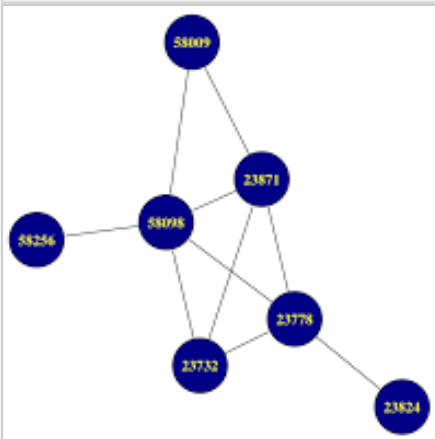
You can also use custom layouts. Here, I'll set up the vertices on a line and then used curved edges.

```
l=matrix(c(1,2,3,4,5,6,7, 1,2,3,4,5,6,7),ncol=2)
plot.igraph(g,layout=l,edge.curved=TRUE)
```

By default, vertices are labeled by their node indexing number (which used to start with 0 in igraph 0.5 but now starts with 1 in igraph 0.6). You can alter the vertex label, vertex color and edge color, etc.:

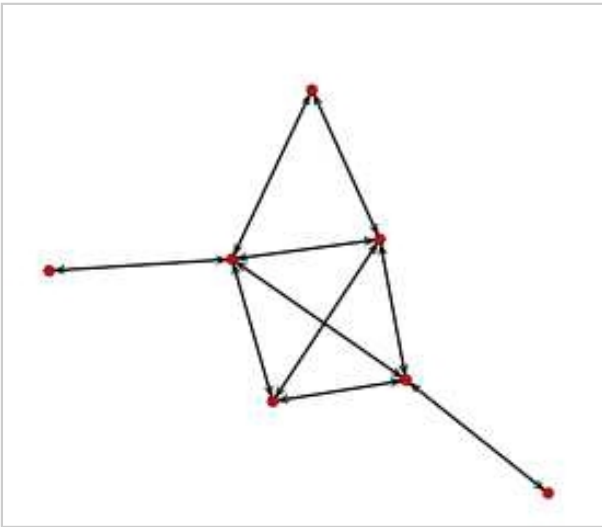
```
plot.igraph(g, vertex.label=V(g)$name, vertex.size=30, vertex.label.color="yellow", vertex.color="darkblue", edge.color="black")
```



For a ton more arguments you can use to make your network look pretty, see **?igraph.plotting**

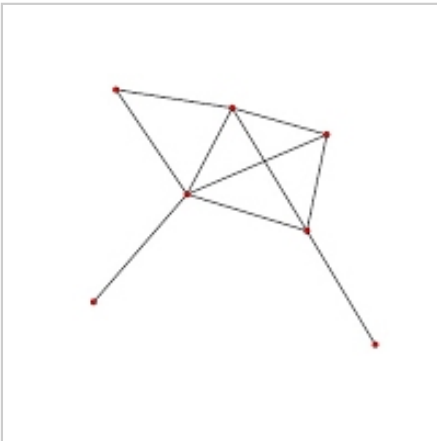
----- In statnet:-----

```
library(statnet)
# import matrix data and convert to a network object. You can find the data I use here in the "importing
data" section - download the sample_adjmatrix.csv file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
net=network(m,matrix.type="adjacency",directed=FALSE)
#open a graphic window and plot the network as an 'undirected' graph
quartz()
gplot(net)
```



... you can see that the default of the `gplot()` command in `statnet` is to produce a “digraph”—that is, a graph with directionality. If you want simple lines rather than arrows, use:

```
quartz()
gplot(net, gmode="graph")
```



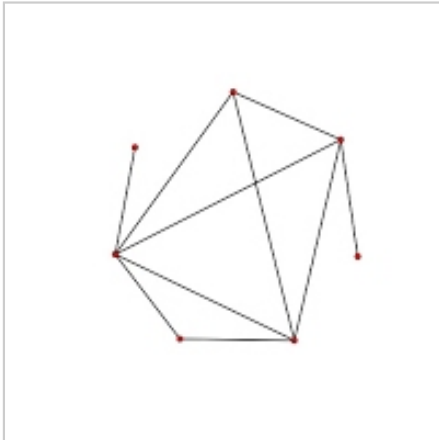
... Notice also that these two plots look a bit different. However, they are identical as far as their network structure goes.

The default algorithm for producing plots in `statnet` is the **Fruchterman and Reingold's (1991)** force-directed placement algorithm. However, there are 17 different algorithms you can use. You can also plot in 3D using the `gplot3d()` command,

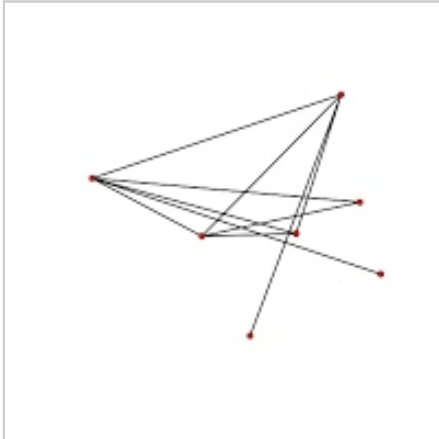
but I have not used it and will not go into it here. You can lookup all of the different layout options by using the `?gplot.layout` help command.

Just as an example, here are two other plots of the exact same network, arranged in a circle and via multidimensional scaling:

```
gplot(net,gmode="graph",mode="circle")
```



```
gplot(net,gmode="graph",mode="mds")
```



More on social networks in R

-

01. igraph vs statnet

-

02. Importing data for social network analysis

-

03. Affiliation Networks/Bipartite Networks

-

04. Node-level Calculations

-

05. Plotting networks - Basics

-

06. Plotting networks: using node attributes

-

07. Plotting Networks: Weighted Edges

-

08. Plotting Networks: Directed Networks

-

09. Edgelist with edge values

-

10. Raw dyadic interaction data

-

Association Indices

-

sexualnetworks

-

Triangle Transitivity in dominance hierarchies & directed graphs

sample_adjmatrix.csv

(0k)

Dai Shizuka,

Apr 27, 2012, 8:40 AM

Comments

[Sign in](#)|[Recent Site Activity](#)|[Report Abuse](#)|[Print Page](#)|Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

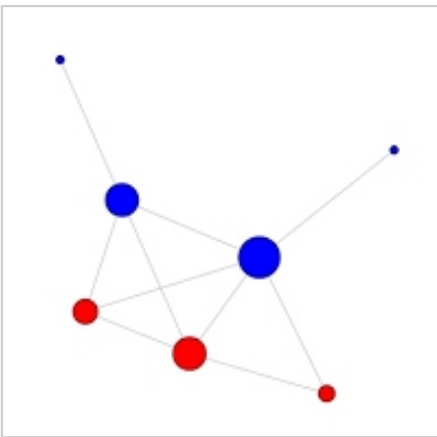
06. Plotting networks: using node attributes

****I tried these codes in igraph v0.6, R 2.15, and all seems to be working fine. Let me know if you have any problems. (DS, Feb 9, 2013)**

In this section, I'll run through methods to represent different qualities of nodes (vertices) in a network plot. I will mainly use igraph. For a full list of plotting arguments in igraph, use:

```
?igraph.plotting
```

Our goal here will be to produce a network figure with different color and size for nodes, like this:



This requires loading *vertex attributes* in the network. In general, when you construct a network you will have some sort of data on the "attributes" of nodes. In our example, we'll use the sex of individuals as the main attribute that determines the color of the nodes. We'll then set the size of the nodes as the degree of the node (i.e., simply the number of edges that is connected to that node).

To do this, we'll first require a list of attributes that are associated with each node. I have supplied this at the bottom of this page as an attachment called "sample_attributes". As with other files, it is a .csv file. It looks like this:

Bird.ID	Sex
23732	F
23871	F
58009	F
23778	M
23824	M
58098	M
58256	M

Notice that I sorted this by sex so that the Bird.ID (i.e. vertex IDs) are not sorted in order. I did this on purpose since you often will not have the attributes file sorted exactly the way the IDs are arranged in the adjacency matrix.

The first thing to note is that in igraph, vertex attributes have the notation `V(graphname)$attributename`

When you load a 'graph object' in igraph, the vertex names are automatically created, so:

```
library(igraph)
#import the sample_adjmatrix file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
net=graph.adjacency(m,mode="undirected",weighted=NULL,diag=FALSE)
# show the names of the vertices you just imported:
V(net)$name

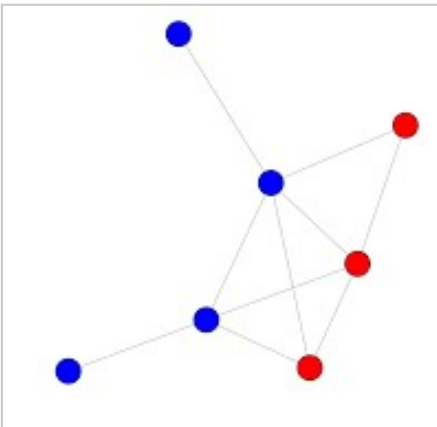
#the result will be:
#[1] "23732" "23778" "23824" "23871" "58009" "58098" "58256"
```

what you want to do, then, is to assign attributes to these vertices. To assign the sex using your attributes file:

```
#import the sample_attributes
a=read.csv(file.choose())
V(net)$Sex=as.character(a$Sex[match(V(net)$name,a$Bird.ID)]) # This code says to create a vertex attribute
called "Sex" by extracting the value of the column "Sex" in the attributes file when the Bird ID number
matches the vertex name.
V(net)$Sex # This will print the new vertex attribute, "Sex"
# and you will get:
#[1] "F" "M" "M" "F" "F" "M" "M"
```

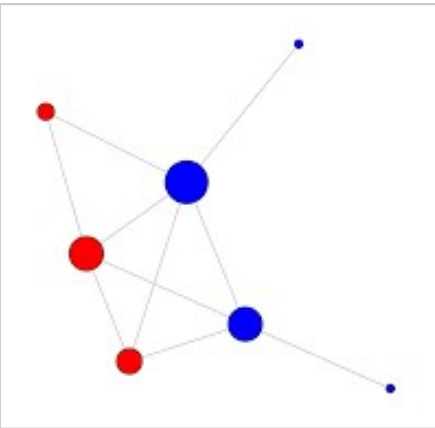
using this, you will then assign colors to the "F" and "M" categories, and move this to a vertex attribute named "color". This is a pre-assigned vertex attribute number. When you fill this, the plot will automatically refer to these values for setting node colors.

```
V(net)$color=V(net)$Sex #assign the "Sex" attribute as the vertex color
V(net)$color=gsub("F","red",V(net)$color) #Females will be red
V(net)$color=gsub("M","blue",V(net)$color) #Males will be blue
plot.igraph(net,vertex.label=NA,layout=layout.fruchterman.reingold)
```



Now, let's play with the node size. Node size is set through a vertex attribute named "size". I'm going to set up the node size as the degree centrality of the nodes.

```
V(net)$size=degree(net)*5 #because 1 is a small size for a node, I'm just multiplying it by 5
plot.igraph(net,vertex.label=NA,layout=layout.fruchterman.reingold)
```

** The ALTERNATIVE way to do this is to set up a vector separately from the 'vertex attribute', and assign the color when you run the plot command. This works equally well.

```
library(igraph)
#import the sample_adjmatrix file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
net=graph.adjacency(m,mode="undirected",weighted=NULL,diag=FALSE)
#import the sample_attributes
a=read.csv(file.choose())
nodecolor=as.character(a$Sex[match(V(net)$name,a$Bird.ID)])
nodecolor=gsub("F","red",nodecolor)
nodecolor=gsub("M","blue",nodecolor)
nodesize=degree(net)*5
plot.igraph(net,vertex.label=NA,layout=layout.fruchterman.reingold, vertex.color=nodecolor,vertex.size=nodesize)
```

for a full list of different vertex attributes you can assign this way, refer to look under ?plot.igraph or follow [this link](#).

Some of the other useful attributes that have pre-assigned role inside the plot.igraph() function:

- vertex.shape= ##
- label.dist= ## ...the distance of the vertex label from the center of the vertex.
- label.degree= ## ...in RADIANS, the angle from the center of the vertex to put the label, so pi/2 would put the label below the vertex.

... similar notation for edge attributes... for example, setting edge.color="black" will make the lines black and perhaps easier to see. Again, for a full list of the different specifications you can make, use ?igraph.plotting.

More on social networks in R

-
- 01. igraph vs statnet

-
- 02. Importing data for social network analysis

-
- 03. Affiliation Networks/Bipartite Networks

-
- 04. Node-level Calculations

-
- 05. Plotting networks - Basics

-
- 06. Plotting networks: using node attributes

-
- 07. Plotting Networks: Weighted Edges

-
- 08. Plotting Networks: Directed Networks

-
- 09. Edgelist with edge values

-
- 10. Raw dyadic interaction data

-
- Association Indices

-
- sexualnetworks

-
- Triangle Transitivity in dominance hierarchies & directed graphs

sample_adjmatrix.csv

(0k)

Dai Shizuka,

Apr 27, 2012, 8:42 AM

sample_attributes.csv

(0k)

Dai Shizuka,

Apr 29, 2010, 10:48 AM

Comments

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

07. Plotting Networks: Weighted Edges

****Feb 9, 2013: I tried running these codes in igraph v0.6, R 2.15 and all seems to be working ok.**

Now that I have described how to add node attributes to network figures, I will introduce how to add *edge values* or *edge weights* to your network. Simply put, we are now moving from a 0-1 adjacency matrix to a matrix that can have a continuous range of values. I illustrate the basic approach in igraph first, and then I show the equivalent codes for statnet below.

For example, consider our hypothetical network of birds--but now some birds are associated more strongly with each other than others. This could be that these individuals are associated in foraging groups more often, or that they interact more frequently with each other than with other individuals--whatever fits your study.

	23732	23778	23824	23871	58009	58098	58256
23732	0	8	0	1	0	10	0
23778	8	0	1	15	0	1	0
23824	0	1	0	0	0	0	0
23871	1	15	0	0	1	5	0
58009	0	0	0	1	0	7	0
58098	10	1	0	5	7	0	1
58256	0	0	0	0	0	1	0

This matrix is available in .csv form at the bottom of the page as an attachment.

... and we'll also make use of the information on sex of individuals as shown before:

Bird.ID	Sex
23732	F
23871	F
58009	F
23778	M
23824	M
58098	M
58256	M

This is again available at the bottom of the page as a .csv file.

Now I'll break down the codes, which are nearly identical to what I have shown in previous pages, save for a few critical differences.

```
library(igraph)
#import the sample_weighted_adjmatrix file from bottom of the page:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # read .csv file
m=as.matrix(dat)
net=graph.adjacency(m,mode="undirected",weighted=TRUE,diag=FALSE) #here is the first difference from the
previous plot: we designate weighted=TRUE
```

That is all that is needed to make a weighted network in igraph: set weighted=TRUE when importing the matrix. We can confirm that the network is weighted simply by calling graph object:

```
summary(net)
```

Which will show this (*in igraph v0.6, the summary output in igraph v0.5 is different):

```
> summary(net)
IGRAPH UNW- 7 10 --
attr: name (v/c), weight (e/n)
```

Note that we now have an "edge attribute" called weight. This means that edges have values, and this has been assigned an object name. To call up the edge values, we simply use:

```
E(net)$weight
```

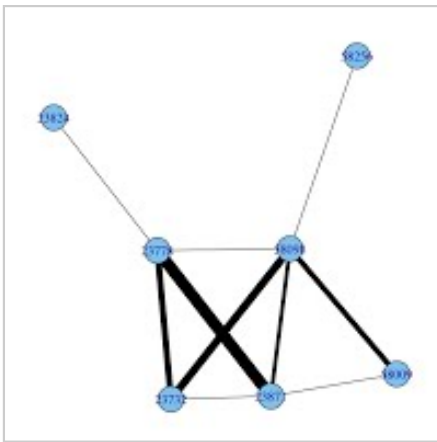
Which will show this:

```
> E(net)$weight
[1] 8 1 10 1 15 1 1 5 7 1
```

So, we simply use E(net)\$weight as part of the options for the network plot:

```
plot.igraph(net,vertex.label=V(net)$name,layout=layout.fruchterman.reingold, edge.color="black",edge.
width=E(net)$weight)
```

This should produce a figure that looks something like this:



Here's the full set of codes to produce a figure of the weighted network, with nodes color coded by sex:

```
library(igraph)
#import the sample_weighted_adjmatrix file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # read .csv file
m=as.matrix(dat)
net=graph.adjacency(m,mode="undirected",weighted=TRUE,diag=FALSE)

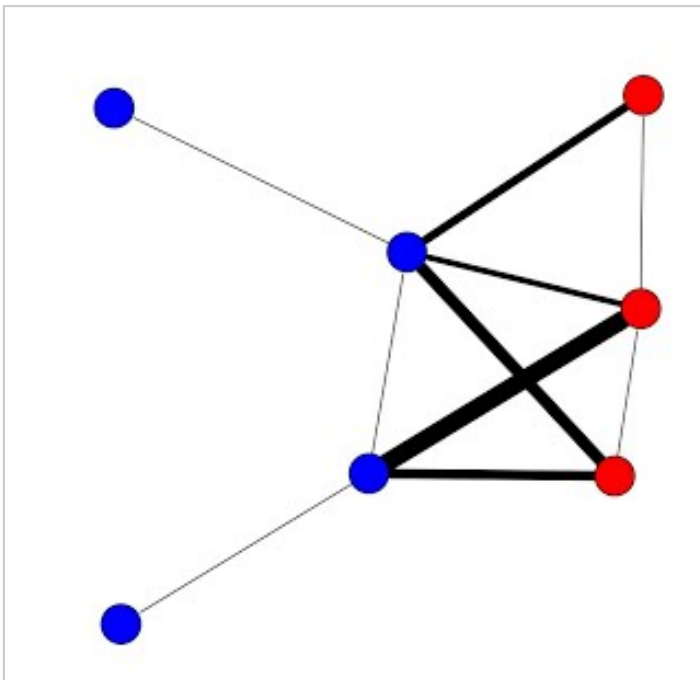
#import the sample_attributes file:
a=read.csv(file.choose())

V(net)$Sex=as.character(a$Sex[match(V(net)$name,a$Bird.ID)])

V(net)$color=V(net)$Sex
V(net)$color=gsub("F","red",V(net)$color)
V(net)$color=gsub("M","blue",V(net)$color)

plot.igraph(net,vertex.label=NA,layout=layout.fruchterman.reingold, edge.color="black",edge.width=E(net)$weight)
```

The color-coding of sexes is identical to what was presented in the previous "using node attributes" section.



... in this hypothetical example, I made the matrix such that edges between males and females are stronger than edges between individuals of the same sex.

***** Weighted edges in Statnet *****

Unfortunately, loading a weighted network in **statnet** is slightly more complicated.

The first step is to create a network object while specifying "ignore edge values = false" and also storing the edge values as an edge attribute:

```
library(statnet)
#import the sample_weighted_adjmatrix file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # read the sample_weighted_adjmatrix.
csv file
m=as.matrix(dat)
net=network(m,matrix.type="adjacency",directed=FALSE,ignore.eval=FALSE,names.eval="value")
# this creates a network object from the matrix, storing the edge values as an edge attribute called
"value"
```

Now you have a network object named "net" that is a weighted network. Here's what the summary(net) function will call:

```
> summary(net)
Network attributes:
vertices = 7
directed = FALSE
hyper = FALSE
loops = FALSE
multiple = FALSE
bipartite = FALSE
total edges = 10
missing edges = 0
```

```

non-missing edges = 10
density = 0.4761905

Vertex attributes:
vertex.names:
  character valued attribute
  7 valid vertex names

Edge attributes:

value:
  numeric valued attribute
  attribute summary:
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.00   1.00   3.00   5.00   7.75  15.00

Network adjacency matrix:
      23732 23778 23824 23871 58009 58098 58256
23732      0      1      0      1      0      1      0
23778      1      0      1      1      0      1      0
23824      0      1      0      0      0      0      0
23871      1      1      0      0      1      1      0
58009      0      0      0      1      0      1      0
58098      1      1      0      1      1      0      1
58256      0      0      0      0      0      1      0
>

```

What you see here is that the 'adjacency matrix' of this network will still show only the 0-1 values. However, an edge attribute 'value' has the edge weights stored. In order to show the sociomatrix with the actual edge weights, you need to use something like this:

```

wnet=as.sociomatrix(net,"value")
wnet

```

which will show:

```

> wnet=as.sociomatrix(net,"value")
> wnet
      23732 23778 23824 23871 58009 58098 58256
23732      0      8      0      1      0     10      0
23778      8      0      1     15      0      1      0
23824      0      1      0      0      0      0      0
23871      1     15      0      0      1      5      0
58009      0      0      0      1      0      7      0
58098     10      1      0      5      7      0      1
58256      0      0      0      0      0      1      0

```

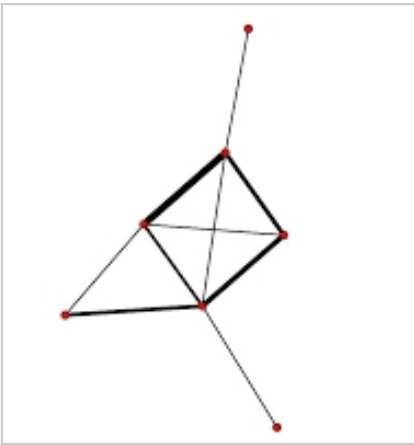
To plot the network with edge weights, use:

```

gplot(net,gmode="graph",edge.lwd=net%e%'value')

```

net%e%'value' in statnet calls the edge attribute named 'value'. (It is equivalent to the notation E(net)\$value in igraph)



Here are the full codes that will replicate the color-coded and weighted network figure in statnet, using the same .csv files:

```
library(statnet)
#import the sample_weighted_adjmatrix file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
net=network(m,matrix.type="adjacency",directed=FALSE,ignore.eval=FALSE, names.eval="value")

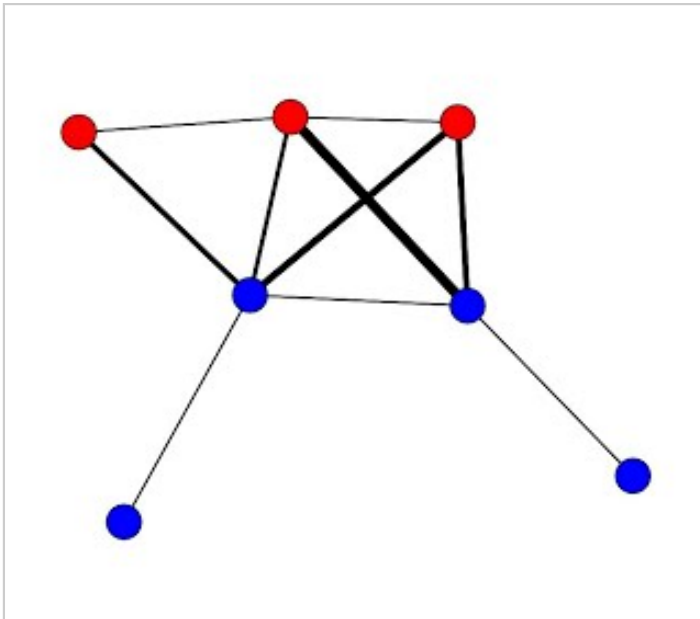
#import the sample_attributes file:
a=read.csv(file.choose())
net%v%'Sex'=as.character(a$Sex[match(net%v%'vertex.names',a$Bird.ID)])

net%v%'color'=net%v%'Sex'
net%v%'color'=gsub("F","red",net%v%'color')
net%v%'color'=gsub("M","blue",net%v%'color')

gplot(net,gmode="graph",mode="fruchtermanreingold", edge.lwd=net%e%'value',vertex.col=net%v%'color',vertex.
cex=3,vertex.sides=20)
```

*note the few unfamiliar options in the gplot() function: vertex.cex is the expansion factor for vertex size. vertex.sides is the number of sides that the vertex has--the default is 8. With about 20, the vertex looks like a circle. Also note that the default plotting algorithm in statnet is the Fruchterman-Reingold method, so the mode="fruchtermanreingold" here is redundant but I include it here for illustration purposes.

Here is the result:



More on social networks in R

- - 01. [igraph vs statnet](#)
- - 02. [Importing data for social network analysis](#)
- - 03. [Affiliation Networks/Bipartite Networks](#)
- - 04. [Node-level Calculations](#)
- - 05. [Plotting networks - Basics](#)
- - 06. [Plotting networks: using node attributes](#)
- - 07. [Plotting Networks: Weighted Edges](#)

-

08. Plotting Networks: Directed Networks

-

09. Edgelists with edge values

-

10. Raw dyadic interaction data

-

Association Indices

-

sexualnetworks

-

Triangle Transitivity in dominance hierarchies & directed graphs

sample_attributes.csv

(0k)

Dai Shizuka,

Jan 9, 2011, 10:32 AM

sample_weighted_adjmatrix.csv

(0k)

Dai Shizuka,

Jan 9, 2011, 10:32 AM

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

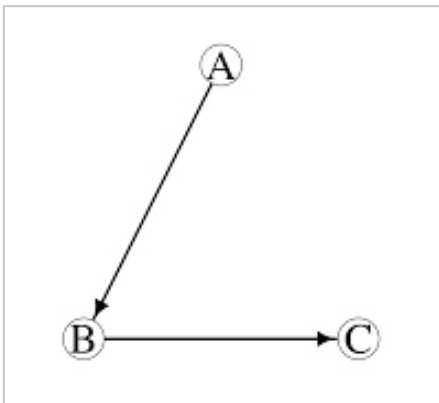
News [R Resources](#) • > • [Social Networks in R](#) • > •

08. Plotting Networks: Directed Networks

****Feb 9, 2013: These codes seem to be working ok in igraph v0.6, R2.15.**

Another major class of networks is a "directed network", aka "directed graph", aka "digraph". Simply, this means that edges have directionality (i.e., are "asymmetric"). In an example that I would use, an individual exhibits aggression to another individual, thus this interaction is directional (i.e., aggression FROM node A TO node B). In such cases, edges would be drawn as an arrow. In this section, I will simply describe how to draw such directed networks in R.

Let's start with what the adjacency matrix will look like. In an undirected network, the adjacency matrix is symmetrical. That is, the values of the matrix are mirror-images across the diagonal. In contrast, the adjacency matrix for an directed network is asymmetrical. Let's demonstrate this with a simple example. Let's say we have a three-node network, and an edge goes from node A to node B, and a second edge goes from node B to node C.



(By the way, this is the code for making the above figure):

```

library(igraph)
g=graph.formula(A-+B,B-+C)
layout=matrix(c(2,1, 1,0, 3,0),byrow=TRUE,nrow=3)
plot.igraph(g, layout=layout,edge.color="black",vertex.color="white", vertex.label=c("A","B","C"),vertex.size=20,vertex.label.cex=3, vertex.label.color="black",edge.width=3,edge.arrow.size=1.2,edge.arrow.width=1.2)
  
```

The adjacency matrix of the above network would look like this:

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

By convention, the origin of the directed edge (i.e., the node that the arrow points from) are the rows, and the recipients of the edges are the columns.

More simply, this network can be represented as an edge list, in which the first column represents the origins of the edges and the nodes on the second column receive the edges:

Node 1	Node 2
A	B
B	C

Ok, now let's up the ante a bit. We will now create an 8-node network with edge weights. This is modified from the section on plotting weighted networks. The following mock data is available as `sample_weighted_directed_adjmatrix.csv` at the **bottom of the page**:

	A	B	C	D	E	F	G
A	0	8	0	1	0	0	0
B	0	0	0	15	0	1	0
C	0	1	0	0	0	0	0

D	1	15	0	0	1	5	0
E	0	0	0	1	0	0	0
F	10	0	0	0	7	0	1
G	0	0	0	0	0	1	0

****NOTE****

I want to point out that there are four possible types of dyadic relationships in a weighted, directed network:

- (1) no interactions -- cells on both sides of the diagonal are zero. Example: A and C
- (2) an unidirectional interaction -- non-zero value on one side of the diagonal, and zero on the other. Example: A and B
- (3) symmetrically mutual interaction -- cells on both sides have an equal, non-zero value. Example: E and D
- (4) asymmetrical mutual interaction -- cells on both sides have non-zero values, but the values are different.

I have conveniently omitted any examples of type 4, the asymmetric mutual interaction. This is because it requires a slightly different graphical representation, where each mutual dyads are connected by two directed edges. I will deal with this later in this section. First, let's just deal with the simple case.

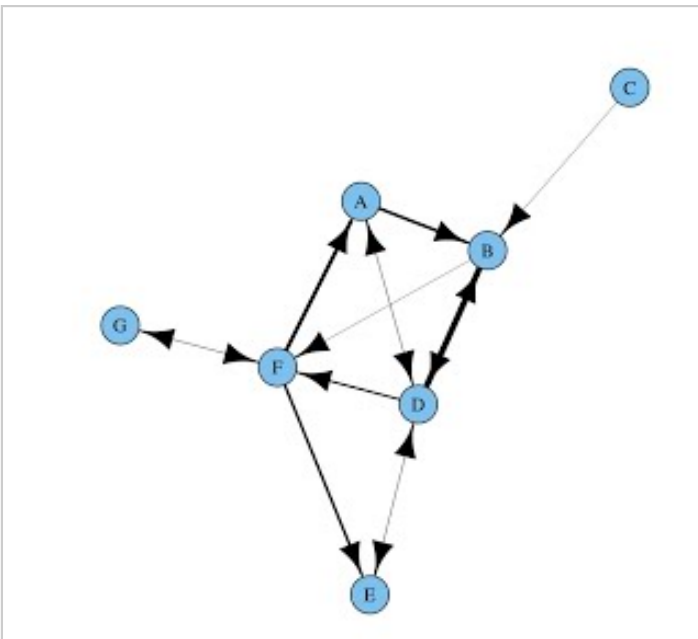
A simple way to draw weighted, directed network:

To plot this weighted, directed network using igraph:

```
library(igraph)
#import the sample_dw_adj.csv file:
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # read .csv file
m=as.matrix(dat)
net=graph.adjacency(m,mode="directed",weighted=TRUE,diag=FALSE) #the only difference between this and the
weighted network code is that mode="directed"

plot.igraph(net,vertex.label=V(net)$name,layout=layout.fruchterman.reingold, vertex.label.color="black",
edge.color="black",edge.width=E(net)$weight/3, edge.arrow.size=1.5)
```

... and you'll get something like this:



You can draw a similar network in statnet:

```

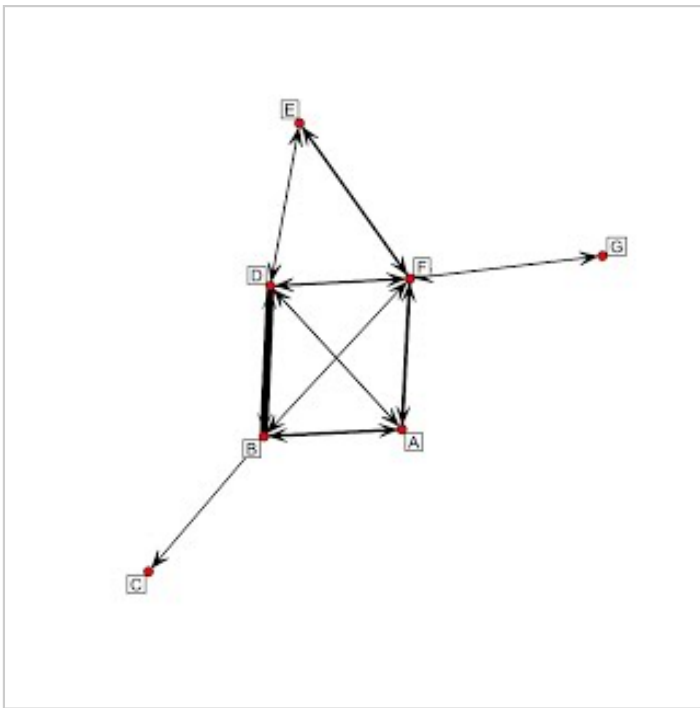
library(statnet)

dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) # read the sample_dw_adj.csv file
m=as.matrix(dat)
net=network(m,matrix.type="adjacency",directed=FALSE,ignore.eval=FALSE, names.eval="value")
# recall that this creates a network object from the matrix, storing the edge values as an edge attribute
# called "value"
wnet=as.sociomatrix(net,"value")
wnet

gplot(net,gmode="digraph",edge.lwd=net%e%'value',label=net%v%'vertex.names', arrowhead.cex=2)

```

The resulting figure will look something like this:



Note that in both `igraph` and `statnet`, the arrowhead sizes are uniform for all edges, so if one of your edges is really thick, it might be difficult to see the arrowhead. You can make all the arrowheads larger, but then your thin edges look weird. I'm not sure what the solution is, except to tinker with the proportional edge widths based on the edge weight you have, and come up with something where the thickest and thinnest edges look fairly normal.

Drawing networks with mutual edges with asymmetrical weights:

This turns out to be very **difficult to do in statnet**, as far as I can tell. This is because the when you create the 'edge values' using `names.eval="value"` (4th code line in above code set), it adds the asymmetrical edges pointing both ways and splits it evenly between the two sides of the adjacency matrix. That is, the default in `statnet` is to create a weighted, undirected network. I have not played around with the codes enough to see how you can keep the asymmetrical information (perhaps using `edgelist?`). This isn't to say that `statnet` can't calculated network metrics with this type of information, but it seems difficult to draw them.

In contrast, the task of dealing with asymmetrical weighted, directed networks seems to be **easier in igraph**. Let's play with this adjacency matrix, available as `sample_weighted_directed_adjmatrix_2` below.

	A	B	C	D	E
A	0	8	0	5	0
B	3	0	0	2	0
C	0	1	0	0	0
D	0	15	0	0	12
E	0	0	0	3	0

The code is basically the same as above, except there is now an `edge.curved=TRUE`:

```
library(igraph)
```

```

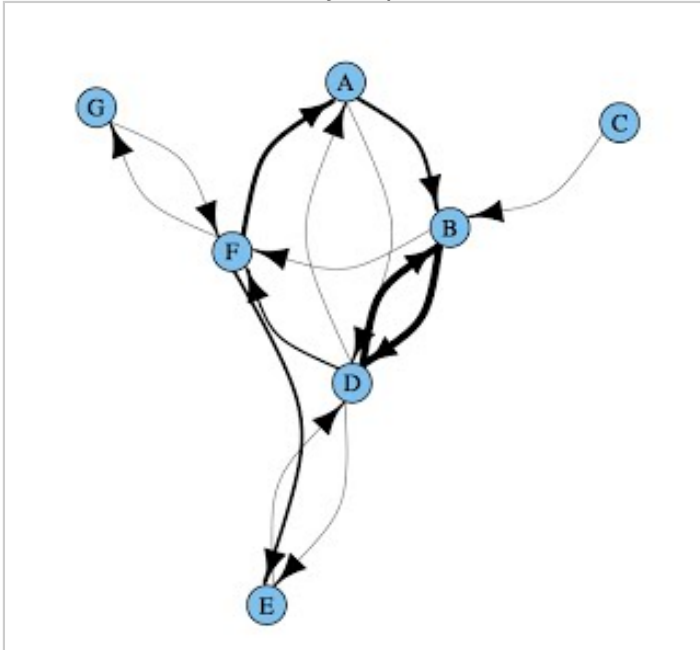
dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE)
m=as.matrix(dat)
net=graph.adjacency(m,mode="directed",weighted=TRUE,diag=FALSE)

plot.igraph(net,vertex.label=V(net)$name,layout=layout.fruchterman.reingold, vertex.label.color="black",
edge.color="black",edge.width=E(net)$weight/3, edge.arrow.size=1.5,edge.curved=TRUE)

```

And this will produce a figure that looks like this.

You can see that when you produce curved directed network plots in igraph, it shows you the asymmetry in edge weights.



More on social networks in R

-
- 01. [igraph vs statnet](#)
-
- 02. [Importing data for social network analysis](#)
-
- 03. [Affiliation Networks/Bipartite Networks](#)
-
- 04. [Node-level Calculations](#)

-
- 05. Plotting networks - Basics
-
- 06. Plotting networks: using node attributes
-
- 07. Plotting Networks: Weighted Edges
-
- 08. Plotting Networks: Directed Networks
-
- 09. Edgelist with edge values
-
- 10. Raw dyadic interaction data
-
- Association Indices
-
- sexualnetworks
-
- Triangle Transitivity in dominance hierarchies & directed graphs

sample_dw_adj.csv

(0k)

Dai Shizuka,

Mar 15, 2015, 7:37 AM

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

09. Edgelists with edge values

***Feb 9, 2013: Codes have been updated for igraph v0.6, R2.15.**

Here, I'll deal with just one topic: using edgelists with associated edge values to create a weighted network.

Let's say you have an edgelist with associated values, like this:

V1	V2	weight
23732	23871	1
23732	58098	10
23778	23732	8
23778	23824	1
23871	23778	15
23871	58009	1
23871	58098	5
58009	58098	7
58098	23778	1
58256	58098	1

This is actually the same data as the weighted adjacency matrix on a [different page](#).

If this network is undirected, then it doesn't matter which vertex is in the 'V1' versus 'V2' columns. However, if the network is directed, then the convention is that the edge goes FROM V1 TO V2.

Also note that each V1-->V2 combo need to be unique. If you have a separate weight for V2-->V1 (e.g. 23871-->58009 is 1 but 58009-->23871 is 3), that's ok. However, if you have two rows that have the same V1-->V2 combo (e.g., 23871-->58009 is 1, and later there is another row that is 23871-->58009 = 3), this will be a problem. So, you either have to sum-up the weights of all duplicate vertex sets before hand, or you need to have each interaction as separate rows (see below). You could, of course, write your own codes to convert whatever format you have to an adjacency matrix to avoid these subtle complications.

The above data is available as a .csv file below ('el.with.weights.csv').

Converting this data into an network is pretty easy. Just remember that if your vertex IDs are numbers like mine, you need to designate them as characters or else igraph/statnet will get confused.

I'll use igraph for this example. Basically all you need to do is create a network using the first two columns using the graph.edgelist() function. Then, you add in the edge weights (third column) to this network:

```
el=read.csv(file.choose()) # read the 'el.with.weights.csv' file
el[,1]=as.character(el[,1]) #Because the vertex IDs in this dataset are numbers, we make sure igraph knows
these should be treated as characters. Otherwise, it'll create problems (see page on data import)
el[,2]=as.character(el[,2])
el=as.matrix(el) #igraph needs the edgelist to be in matrix format
g=graph.edgelist(el[,1:2]) #We first create a network from the first two columns, which has the list of
vertices
E(g)$weight=as.numeric(el[,3]) #We then add the edge weights to this network by assigning an edge
attribute called 'weight'.
```

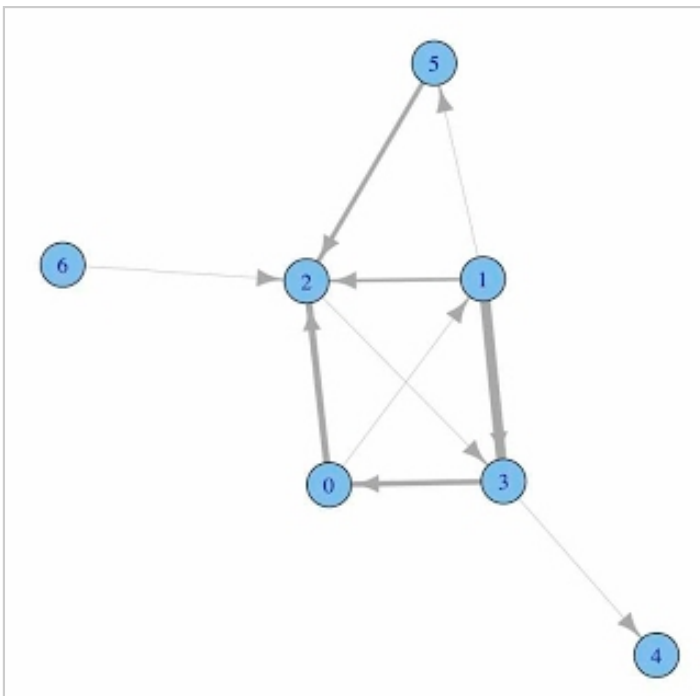
****OR, a much easier way to import a weighted edgelist is to just use the graph.data.frame() function:**

```
el=read.csv(file.choose()) # read the 'el.with.weights.csv' file
g2=graph.data.frame(el)
```

... because the third column is called "weight", this function adds an edge attribute called "weight" when creating the graph. You can have additional edge attributes as the 4th, 5th, etc. columns.

Note that the default for graph.edgelist() is to create a directed network, so the plot will look like this:

```
plot(g, layout=layout.fruchterman.reingold, edge.width=E(g)$weight/2)
```



To get the weighted adjacency matrix from the network:

```
adj=get.adjacency(g,attr='weight') #attr='weight' makes sure that the weights are shown in the adjacency matrix.
adj
```

```
> adj
7 x 7 sparse Matrix of class "dgCMatrix"
      23732 23871 58098 23778 23824 58009 58256
23732      .      1     10      .      .      .
23871      .      .      5     15      .      1
58098      .      .      .      1      .      .
23778      8      .      .      .      1      .
23824      .      .      .      .      .      .
58009      .      .      7      .      .      .
58256      .      .      1      .      .      .
```

This is the "sparse" display of the adjacency matrix. Zeros are displayed as periods so that you can visually pick out the non-zero values. If you don't like this, you can use:

```
adj=get.adjacency(g,attr='weight',sparse=FALSE)
adj
```

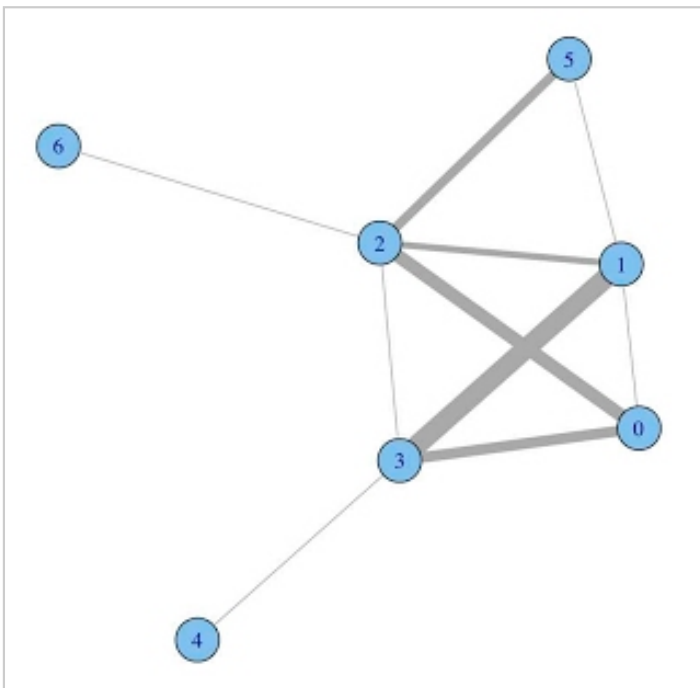
```
> adj
      23732 23871 58098 23778 23824 58009 58256
23732      0      1     10      0      0      0
23871      0      0      5     15      0      1
58098      0      0      0      1      0      0
23778      8      0      0      0      1      0
23824      0      0      0      0      0      0
58009      0      0      7      0      0      0
58256      0      0      1      0      0      0
```

238 / 1	0	0	5	15	0	1	0
58098	0	0	0	1	0	0	0
23778	8	0	0	0	1	0	0
23824	0	0	0	0	0	0	0
58009	0	0	7	0	0	0	0
58256	0	0	1	0	0	0	0

If you want the undirected network, then you simply use **g=graph.edgelist(el[,1:2],directed=FALSE)** when importing the data, and the rest is the same, like so:

```
el=read.csv(file.choose()) # read the 'el.with.weights.csv' file
el[,1]=as.character(el[,1])
el[,2]=as.character(el[,2])
el=as.matrix(el)
g=graph.edgelist(el[,1:2],directed=FALSE)
E(g)$weight=as.numeric(el[,3])

plot(g, layout=layout.fruchterman.reingold, edge.width=E(g)$weight)
```



Then, the adjacency matrix you get from the network should be the same as those presented in the 'plotting weighted network' page (albeit with different ordering of vertex IDs).

	23732	23871	58098	23778	23824	58009	58256
23732	0	1	10	8	0	0	0
23871	1	0	5	15	0	1	0
58098	10	5	0	1	0	7	1
23778	8	15	1	0	1	0	0
23824	0	0	0	1	0	0	0
58009	0	1	7	0	0	0	0
58256	0	0	1	0	0	0	0

More on social networks in R

- [01. igraph vs statnet](#)

- [02. Importing data for social network analysis](#)

- [03. Affiliation Networks/Bipartite Networks](#)

- [04. Node-level Calculations](#)

- [05. Plotting networks - Basics](#)

- [06. Plotting networks: using node attributes](#)

- [07. Plotting Networks: Weighted Edges](#)

- [08. Plotting Networks: Directed Networks](#)

- [09. Edgelists with edge values](#)

- [10. Raw dyadic interaction data](#)

- [Association Indices](#)
- [sexualnetworks](#)
- [Triangle Transitivity in dominance hierarchies & directed graphs](#)

el.with.weights.csv

(0k)

Dai Shizuka,

Feb 24, 2012, 12:31 PM

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

10. Raw dyadic interaction data

Feb 9, 2013: These codes have been updated for igraph v0.6 in R 2.15.

This page is somewhat related to the ['edgelists with associated edge values' page](#):

Here, I will address how we go from the type of data I often deal with--a list of outcomes of interactions, whereby the same combination of individuals might interact multiple times over time. The data format look like something like a typical edgelist, except that each row represents a single *interaction* instead of a dyadic *relation* (which is the sum total of all observed interactions).

So the question is, how can we efficiently convert data on a set of interactions to a network of relations.

For example:

Winner	Loser
23871	23778
23871	58098
23778	23824
23732	58098
23871	23778
23778	23732

```

23871 23778
23871 58009
23732 58098
58009 58098
23778 23732
23871 58098
23732 58098
23871 23778
23732 58098
23732 58098
. .
. .
. .

```

The full data is available below as 'interactions.csv'. This is actually an alternate format of the 'weighted edgelist' from the [previous page](#).

For simplicity, this dataset does not contain any 'reversals'... but we'll deal with data with reversals a bit later.

This type of data can be converted easily to a network. Here's an example in igraph:

```

library(igraph)
el=as.matrix(read.csv(file.choose()))
el[,1]=as.character(el[,1])#Because the vertex IDs in this dataset are numbers, we make sure igraph knows
these should be treated as characters. Otherwise, it'll create problems.
el[,2]=as.character(el[,2])
g=graph.edgelist(el)

a=get.adjacency(g,sparse=FALSE)
a

```

	23871	23778	58098	23824	23732	58009	58256
23871	0	15	5	0	0	1	0
23778	0	0	0	1	8	0	0
58098	0	1	0	0	0	0	0
23824	0	0	0	0	0	0	0
23732	1	0	10	0	0	0	0
58009	0	0	7	0	0	0	0
58256	0	0	1	0	0	0	0

This should be the same as the same adjacency matrix we get in the 'edgelist with edge values' page, although the order of the vertices are different.

****HOWEVER**, there is one problem with this network. Even though the adjacency matrix suggests that the edge weight information is included in this igraph object, there in fact is no edge weight here. This is clear if we just look at the summary information for g.

```
summary(g)
```

```

> summary(g)
IGRAPH DN-- 7 50 --

```

```
attr: name (v/c)
```

This network does not have edge weights.

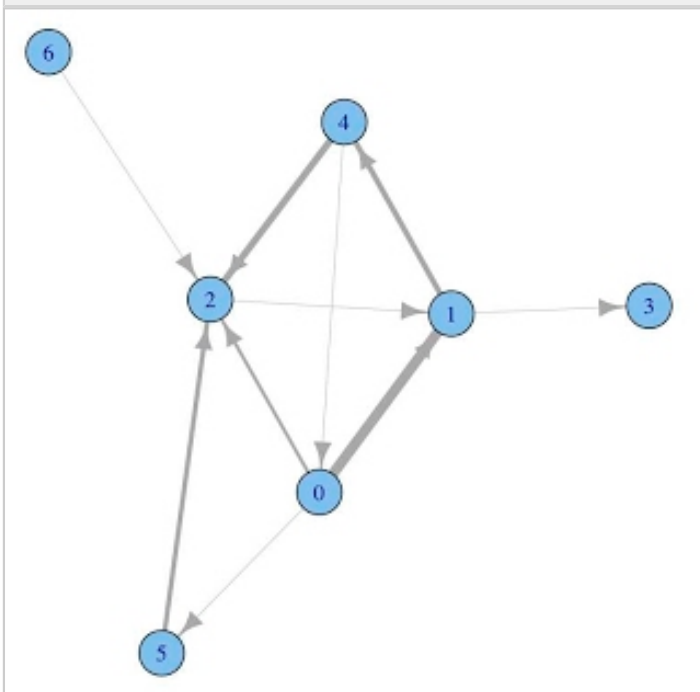
So, to make a network that actually incorporates edge weights, we have to go from the adjacency matrix (which, as shown above, has the weight values) and make a new igraph object:

```
a=get.adjacency(g)
g.weight=graph.adjacency(a,mode="directed",weighted=TRUE)
summary(g.weight)
```

```
IGRAPH DNW- 7 10 --
attr: name (v/c), weight (e/n)
```

Ok, so now we have what we want--a network that summarizes the cumulative interactions of all dyads. So if we plot this figure:

```
plot(g.weight,edge.width=E(g.weight)$weight/2)
```



This is identical to the [previous page](#).

Reversals

Ok, now let's deal with reversals.

First, we'll switch up the dataset a little bit to create some reversals. We'll just flip the winners & losers of 10 out of the 50 interactions.

```

n=sample(nrow(el),10) #randomly sample 10 numbers between 1 through [number of rows in el] (in this case,
50).
el2=el
el2[n,]=el[n,c(2:1)] #for those randomly selected 10 rows, swap columns 1 & 2

g.alt=graph.edgelist(el2) #create a network from this altered interaction list
a.alt=get.adjacency(g.alt,sparse=FALSE) #the adjacency matrix of this alternate network
a.alt

```

	23871	23778	58098	23824	23732	58009	58256
23871	0	11	5	0	0	1	0
23778	4	0	0	1	6	0	0
58098	0	1	0	0	2	2	0
23824	0	0	0	0	0	0	0
23732	1	2	8	0	0	0	0
58009	0	0	5	0	0	0	0
58256	0	0	1	0	0	0	0

Here, we see that, for example, #23871 beat #23778 11 times, but #23778 beat #23871 4 times.

Now, here is the crux of the problem--given that two individuals trade-off winning & losing, *what is the relation* between these two individuals? The simplest way is to say 'the individual that wins a majority of interaction is dominant'. This is clearly very simplistic, but it's probably the easiest and the possible alternatives are likely just as arbitrary...

... so, let's just go with 'majority wins'. Then, we need to convert this matrix of interactions into a matrix that actually represents dominance relations, based on cumulative interactions.

Here's one way to do that:

```

interactions=a.alt+t(a.alt) #the number of interactions between dyads is the sum of the current matrix and
the transpose of the matrix

m=a.alt/interactions #we then divide the number of wins by the corresponding number of interactions
m[m>0.5]=1 #if you win >50% of interactions with this guy, you get a 1
m[m<0.5]=0 #if you win <50% of interactions with this guy, you get a 0
m[is.na(m)]=0 #NaNs mean there were no interactions for this dyad, so they both get 0s
m[m==0.5]=0 #If they win equal number of interactions, the dyad is tied... what you do with these kind
of depends on your question. You can either set it to 0 because the relation is 'unresolved', or you can
set is to 0.5 because they 'split the difference', or you can set both to 1 because they are 'mutual
winners'. Here, ties = 0.
m

```

	23871	23778	58098	23824	23732	58009	58256
23871	0	1	1	0	0	1	0
23778	0	0	0	1	1	0	0
58098	0	1	0	0	0	0	0
23824	0	0	0	0	0	0	0
23732	1	0	1	0	0	0	0
58009	0	0	1	0	0	0	0
58256	0	0	1	0	0	0	0

...as I wrote on the code annotations, you have to set up a guideline for yourself about what to do with 'non-interactions' and

'ties'. The other obvious flaw is that we lose all information about the proportion of times an individual won, etc... but these issues are beyond the scope of this little page.

More on social networks in R

-
- 01. [igraph vs statnet](#)
-
- 02. [Importing data for social network analysis](#)
-
- 03. [Affiliation Networks/Bipartite Networks](#)
-
- 04. [Node-level Calculations](#)
-
- 05. [Plotting networks - Basics](#)
-
- 06. [Plotting networks: using node attributes](#)
-
- 07. [Plotting Networks: Weighted Edges](#)
-
- 08. [Plotting Networks: Directed Networks](#)
-
- 09. [Edgelist with edge values](#)
-
- 10. [Raw dyadic interaction data](#)
-
- [Association Indices](#)
-

[sexualnetworks](#)

-

[Triangle Transitivity in dominance hierarchies & directed graphs](#)

interactions.csv

(1k)

Dai Shizuka,

Feb 24, 2012, 12:55 PM

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

Association Indices

In animal social network studies, it is common to use the "gambit of the group": i.e., building networks based on the presence of individuals at the same place at the same time. For example, you might consider individuals to be 'associated' if they are seen in the same flock at a given time. You have to have an operational definition of what a flock is.

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By [Google Sites](#)

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News

[R Resources](#) • > • [Social Networks in R](#) • > •

sexualnetworks

Comments

[Sign in](#) | [Recent Site Activity](#) | [Report Abuse](#) | [Print Page](#) | Powered By **Google Sites**

Shizuka Lab

School of Biological Science
University of Nebraska-Lincoln

Contact:

Dr. Daizaburo Shizuka
410B Manter Hall
Biological Sciences
University of Nebraska
Lincoln, NE 68588 USA
dshizuka2@unl.edu

- [Shizuka Lab Home](#)

- [People](#)

- [Research](#)

- [Publications](#)

- [R Resources](#)

- [Teaching](#)

- [News](#)

News [R Resources](#) • > • [Social Networks in R](#) • > •

Triangle Transitivity in dominance hierarchies & directed graphs

The study of dominance hierarchies dates back to the 1920s, when Schjelderup-Ebbe (1922) first described the emergence of linear dominance hierarchies in flocks of chickens (this is also the origin of the expression 'peck order' or 'pecking order'). Subsequently, mathematicians like Kendall & Babington Smith (1940) and Landau (1951) derived formulas to describe the structures of such hierarchies (for a good overview of these methods, see de Vries (1995--Animal Behaviour) as well as Appleby (1983--Animal Behaviour)).

The methods by Kendall and Landau, however, were derived in the context of *tournaments*--a specific type of networks in which all asymmetric dyadic relations (e.g., dominant-subordinate) are known. This contrasts with most data on animal dominance hierarchies. Most of the time, there are pairs that are not observed to interact. To deal with this problem, various methods have been used to 'fill in' these unknown relations. The most common method was pioneered by de Vries (1995), in which randomization procedures are used to fill in unknown relations. Linearity indices, such as Landau's h and Kendall's K can then be calculated using these 'filled' tournaments.

However, we (Shizuka & McDonald 2012) recently showed that randomization procedures to fill in dominance data sets can obscure true patterns of transitivity of dominance relations that actually occurred. In this study, we introduced a measure we called 'triangle transitivity', based on the proportion of actual three-way relations that were established (i.e., triangles) that were transitive (i.e., if A beats B and B beats C, then A also beats C). We show that the random expectation of this proportion is 0.75, no matter the size or density of the network. Using this, we can evaluate how orderly dominance relations are, and whether

these values correspond to existing linearity measures.

The following codes will take a dominance data set (in the form of binary dominant-subordinate relations in a matrix) and generate the relevant indices (P_t : the proportion of triangles that are transitive, and t_{tri} : this proportion scaled so that it is 0 for the random expectation and 1 for maximum transitivity). It also conducts a 'conditional uniform graph test' (think of it as a Monte Carlo procedure for testing network metrics) to test whether the given dominance set is more orderly than expected by chance.

This set of codes is published as supplemental materials in this study:

Shizuka, D. and D. B. McDonald. 2012. A social network perspective on transitivity and linearity in dominance hierarchies.

Animal Behaviour. DOI:[10.1016/j.anbehav.2012.01.011](https://doi.org/10.1016/j.anbehav.2012.01.011)

You can (hopefully) find all the details you need in that paper. Please email me if you would like me to send you the pdf, and please cite the paper if you end up using these codes in your work.

UPDATE (9/30/2014)

I have updated this script in two ways: (i) You can now start the script using a raw interaction matrix--i.e., you don't have to convert the matrix to binary dominants=1 and subordinates=0; (2) I incorporated the changes from the [errata in Animal Behavior](#), in which I point out that randomizations of very sparse dominance networks can result in some networks that contain no triangles (i.e., triangle transitivity = NA). The updated script will throw out such no-triangle random networks. Special thanks to James Curley for pointing out a simple way to convert interaction matrices into dominance (0/1) matrices.

Here are the codes:

```
library(statnet)

dat=read.csv(file.choose(),header=TRUE,row.names=1,check.names=FALSE) #start with raw interaction matrix,
with rows as winners/aggressors and columns as losers/receivers

int.to.dom=function(x){ ((x>t(x)) & (x+t(x)>0))+0}

m=int.to.dom(as.matrix(dat))

g=network(m,directed=TRUE)

# We calculate P.t and t.tri for this empirical network.

tri=triad.census(g) #The full triad census as an 16-element vector

w=as.vector(c(0,0,0,0,0,0,0,0,0,1,0,0,1,1,0.5,0.75,0.75)) # The weighting vector for transitivity

N.triangle=sum(tri*as.vector(c(0,0,0,0,0,0,0,0,0,1,1,0,1,1,1,1,1))) #Count and sum the number of triangles

Pt=sum(tri*w)/N.triangle

t.tri=4*(Pt-0.75)
```

Pt

t.tri

We now conduct 1,000 simulations of random networks with the same number of mutual, asymmetric and null dyads as the empirical dominance matrix. We calculate Pt from each of these simulated networks (r.P.t). I use a while() loop because randomizations of some sparse matrices will produce networks with no triangles. For simplicity, we just throw these out.

dyads=dyad.census(g)

r.p.t=vector(length=1000)

j=1

while(j<1001){

r=rguman(1,nv=nrow(m),mut=dyads[1],asym=dyads[2],null=dyads[3])

r.triad=triad.census(r)

r.p.t[j]=r.triad[9]/(r.triad[10]+r.triad[9])

if (is.na(r.p.t[j])) next else j=j+1

}

p=length(r.p.t[r.p.t>=Pt])/1000

p

Comments